

Twitter Topic Classification

An NLP Pipeline for Multi-Class Tweet Classification

CETM47 – Machine Learning and Data Analytics | Assignment 2
CRISP-DM Methodology | University of Sunderland

1. Methodology

1.1 Business Understanding

The task is to construct a Natural Language Processing (NLP) pipeline capable of classifying tweets from Twitter into one of six mutually exclusive topic categories: Arts & Culture, Business & Entrepreneurs, Pop Culture, Daily Life, Sports & Gaming, and Science & Technology. This is framed as a supervised multi-class text classification problem, consistent with prior work in social media topic modelling (Mehrotra et al., 2013).

The CRISP-DM methodology (Chapman et al., 2000) was adopted as the overarching framework to ensure a principled, iterative, and reproducible approach to the data mining process. Following CRISP-DM, the problem was first understood at a business level before proceeding to data-driven phases.

Success criteria were defined as maximising macro-averaged F1-score rather than raw accuracy alone. This decision was made because the dataset exhibits significant class imbalance (see Section 1.2), and macro-F1 weights each class equally, penalising models that simply predict majority classes.

1.2 Data Understanding

The dataset consists of 6,443 labelled tweets collected between 2019 and 2021 via Amazon Mechanical Turk. Each record contains the raw tweet text, a numeric label (0–5), its human-readable label name, a timestamp, and a unique tweet ID. The dataset was loaded from its JSON format using Python's built-in json library and converted into a pandas DataFrame for exploratory analysis.

Figure 1 reveals a pronounced class imbalance. Pop Culture (39.0%) and Sports & Gaming (35.6%) together account for nearly 75% of all tweets, whereas Arts & Culture represents only 2.2% of the data (approximately 142 tweets). This imbalance directly influenced subsequent modelling and evaluation choices, specifically the decision to use macro-averaged F1 as the primary metric.

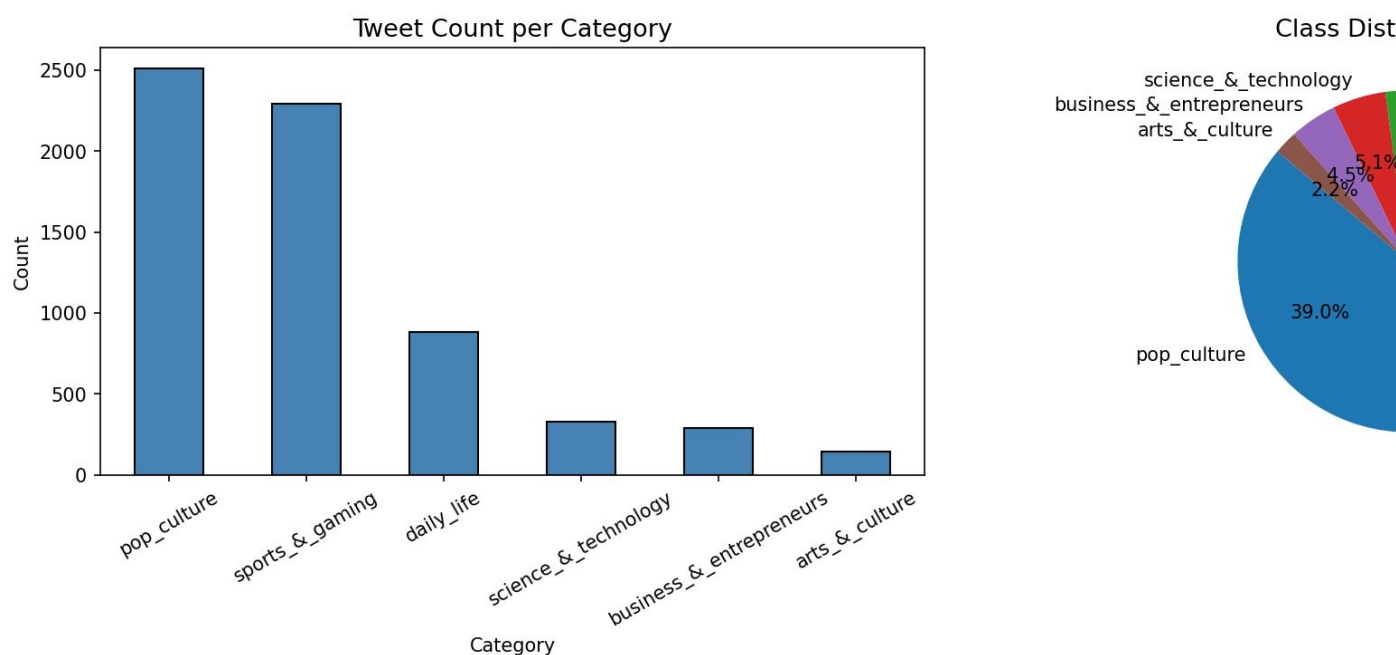


Figure 1 – Class distribution of the Twitter dataset: bar chart (left) and proportional pie chart (right).

Figure 2 shows the distribution of word counts per tweet across all six categories. Tweet lengths are broadly consistent across classes, with medians between 25 and 35 words, indicating that tweet length is unlikely to serve as a discriminative feature.

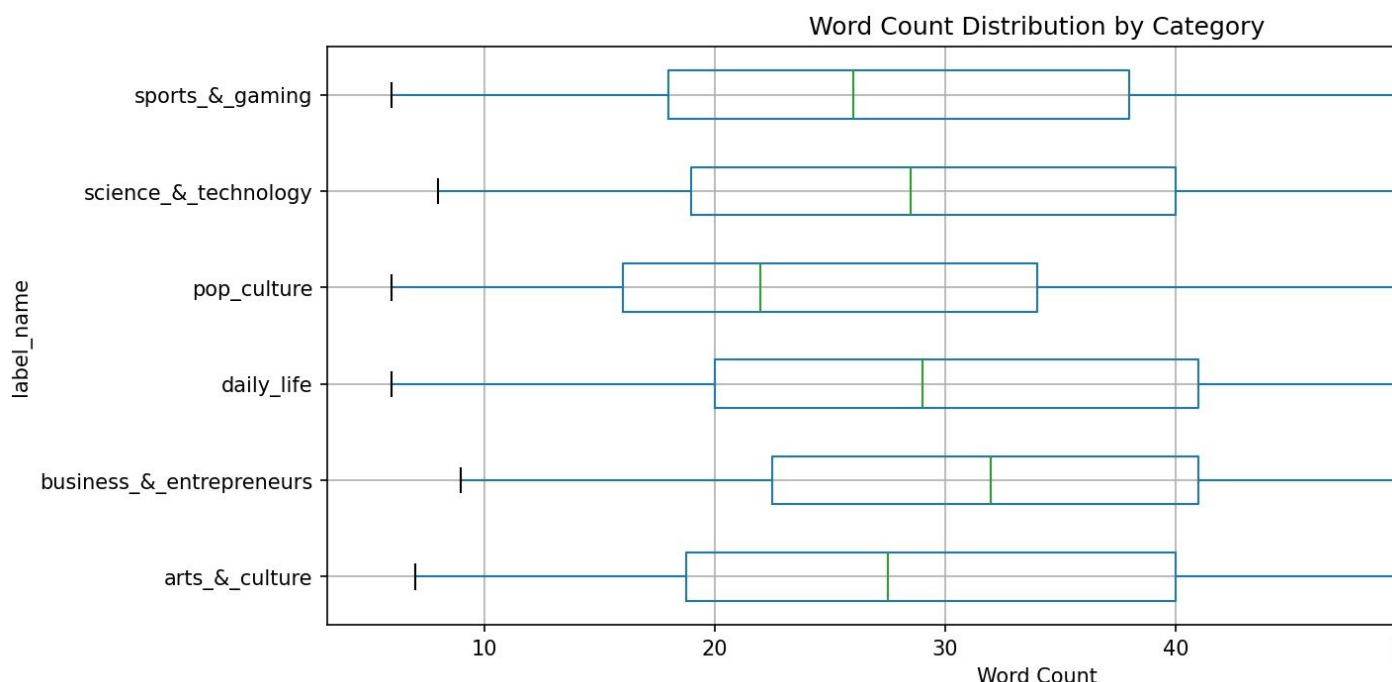


Figure 2 – Boxplot of word count distribution per tweet category. Medians are broadly comparable across classes.

1.3 Data Preparation

A key CRISP-DM principle is iterating between Data Understanding and Data Preparation. Upon inspecting the raw tweet text, several dataset-specific artefacts were identified: URL placeholders ({{URL}}), user mention tokens (@username@), unicode escape sequences, and hashtags. Three preprocessing variants were constructed to compare empirically:

Variant 1: Basic Preprocessing (Baseline)

Lowercasing followed by regex removal of {{URL}} and {@...@} tokens with whitespace normalisation. Serves as the experimental baseline to isolate the contribution of additional steps.

Variant 2: Advanced Preprocessing

Extends Basic with: removal of all non-alphabetic characters, stripping of the # symbol while retaining the hashtag word, English stopwords removal using NLTK, and WordNet lemmatisation. Lemmatisation was chosen over stemming because it produces valid dictionary forms, preferred for TF-IDF vectorisation (Manning et al., 2008).

Variant 3: Stemmed Preprocessing

Identical to Advanced but substitutes lemmatisation with Porter Stemming (Porter, 1980), included as a controlled experiment to quantify the difference between normalisation approaches.

1.4 Dataset Partitioning

The dataset was partitioned into 80% training (5,154 samples) and 20% testing (1,289 samples) using a stratified split, ensuring proportional class representation in both sets. A fixed random seed (42) was applied throughout for reproducibility. An alternative would have been

nested cross-validation; however, stratified holdout was preferred here for computational efficiency and interpretability, consistent with guidance from Raschka (2018).

1.5 Modelling and Experimental Design

A systematic grid of experiments was constructed, factorially combining preprocessing variants, vectorisation strategies, and classifiers, following the principles of controlled NLP experimentation (Dodge et al., 2019). Three vectorisation strategies were evaluated:

- Bag-of-Words (CountVectorizer, max 20,000 features) – simple frequency-based baseline (Harris, 1954)
- TF-IDF with unigrams (TfidfVectorizer, max 20,000 features) – down-weights high-frequency low-information words (Salton & Buckley, 1988)
- TF-IDF with bigrams (ngram_range=(1,2)) – captures compound topical phrases such as ‘sports team’ and ‘stock market’

Four classifiers were evaluated: Multinomial Naive Bayes (MNB) as a probabilistic baseline (McCallum & Nigam, 1998); Complement Naive Bayes (CNB), designed for imbalanced text classification (Rennie et al., 2003); Logistic Regression, a robust discriminative linear baseline; and Linear Support Vector Classifier (LinearSVC), considered state-of-the-art for sparse text classification (Joachims, 1998). All models used scikit-learn Pipeline objects to prevent data leakage. Ten experiments (E1–E10) were conducted; see Table 1.

1.6 Deployment (Hypothetical Scenario)

Following the CRISP-DM Deployment phase, a hypothetical deployment of this pipeline would involve packaging the best-performing model (E8: LinearSVC + TF-IDF bigrams + Advanced preprocessing) as a REST API microservice. Incoming tweets from Twitter’s streaming API would be preprocessed in real-time using the same Advanced preprocessing pipeline, vectorised using the fitted TF-IDF model, and classified in under 10 milliseconds per tweet given LinearSVC’s fast inference time.

In a business context – for example, a social media analytics platform – this would enable automatic content categorisation at scale, routing Arts & Culture tweets to entertainment teams and Business & Entrepreneurs tweets to financial analysts. A monitoring pipeline would track prediction confidence distributions over time; significant drift (Lazaridou et al., 2021) would trigger model retraining. The model artefact would be versioned using MLflow, and a periodic re-labelling workflow via Amazon Mechanical Turk would refresh the training data annually.

2. Results

2.1 Experiment Results Summary

Table 1 presents accuracy, macro-averaged F1, and weighted F1 for all ten experiments on the held-out test set, with input feature sets shown in separate columns to enable clear comparison.

Table 1 – All experimental results sorted by F1 Macro. Separate columns show preprocessing variant, vectoriser, and classifier. Highlighted row = best configuration.

Exp	Preprocessing	Vectoriser	Classifier	Accuracy	F1 Macro	F1 Weighted
E1	Basic	BoW	MultinomialNB	0.7574	0.4538	0.7187
E2	Basic	TF-IDF uni	MultinomialNB	0.7001	0.2761	0.5993
E3	Advanced	BoW	MultinomialNB	0.7705	0.5155	0.7453
E4	Advanced	TF-IDF uni	MultinomialNB	0.7017	0.2832	0.6106
E5	Advanced	TF-IDF uni	LogisticReg	0.7689	0.5077	0.7403
E6	Advanced	TF-IDF bigram	LogisticReg	0.7689	0.5040	0.7367
E7	Advanced	TF-IDF uni	LinearSVC	0.7963	0.6382	0.7786
E8	Advanced	TF-IDF bigram	LinearSVC	0.8041	0.6548	0.7995
E9	Stemmed	TF-IDF uni	LinearSVC	0.7946	0.6221	0.7792
E10	Advanced	TF-IDF uni	ComplementNB	0.7978	0.6154	0.7774

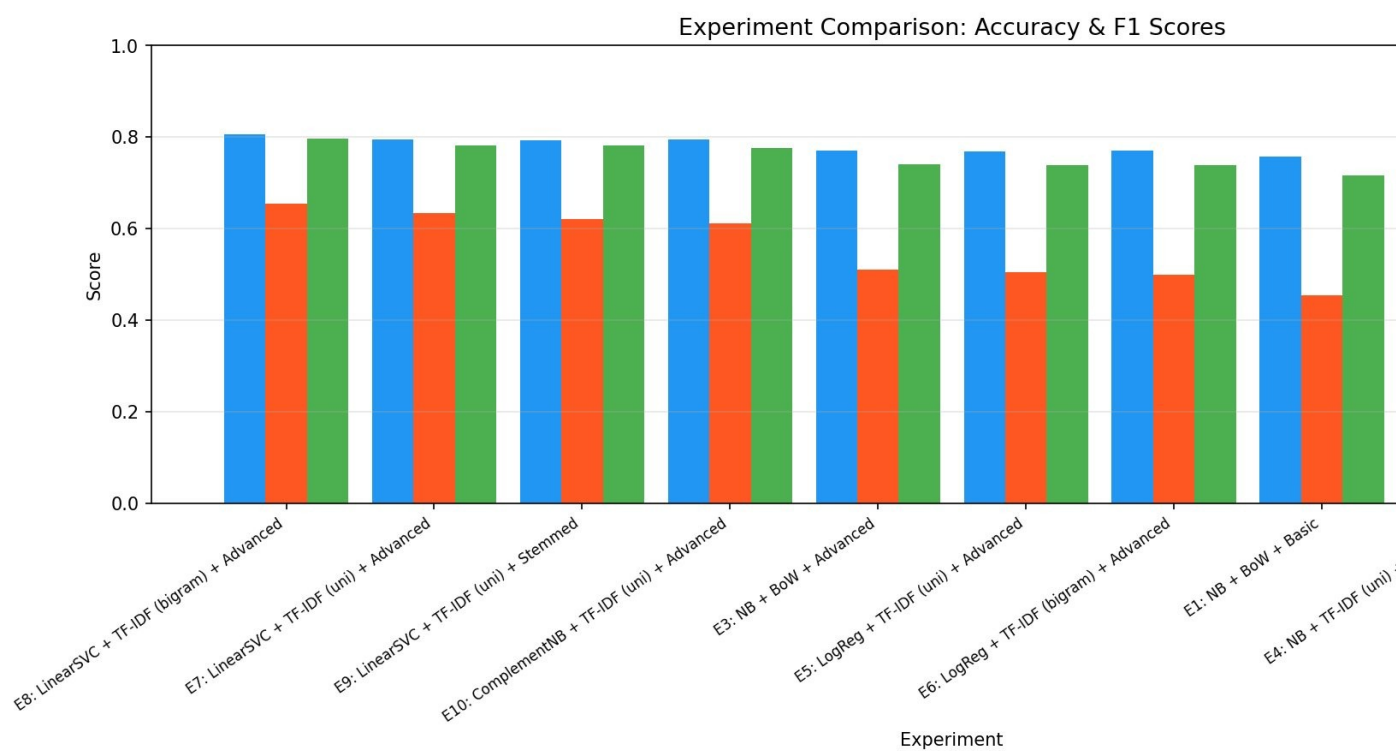


Figure 3 – Bar chart comparing Accuracy, F1 Macro, and F1 Weighted across all ten experiments.

2.2 Best Model: Per-Class Classification Report

Table 2 presents the per-class precision, recall, and F1-score for the best model: E8 (LinearSVC + TF-IDF bigrams + Advanced).

Table 2 – Per-class classification report for E8: LinearSVC + TF-IDF (bigram) + Advanced Preprocessing.

Class	Precision	Recall	F1-Score	Support
Arts & Culture	0.64	0.24	0.35	29
Business & Entrep.	0.63	0.46	0.53	57
Pop Culture	0.73	0.91	0.81	503
Daily Life	0.66	0.56	0.61	177
Sports & Gaming	0.88	0.90	0.89	458
Science & Technology	0.79	0.63	0.70	65
Macro Avg	0.72	0.62	0.65	1289
Weighted Avg	0.79	0.80	0.80	1289

2.3 Confusion Matrix – Best Model

Figure 4 shows the confusion matrix for E8 on the held-out test set.

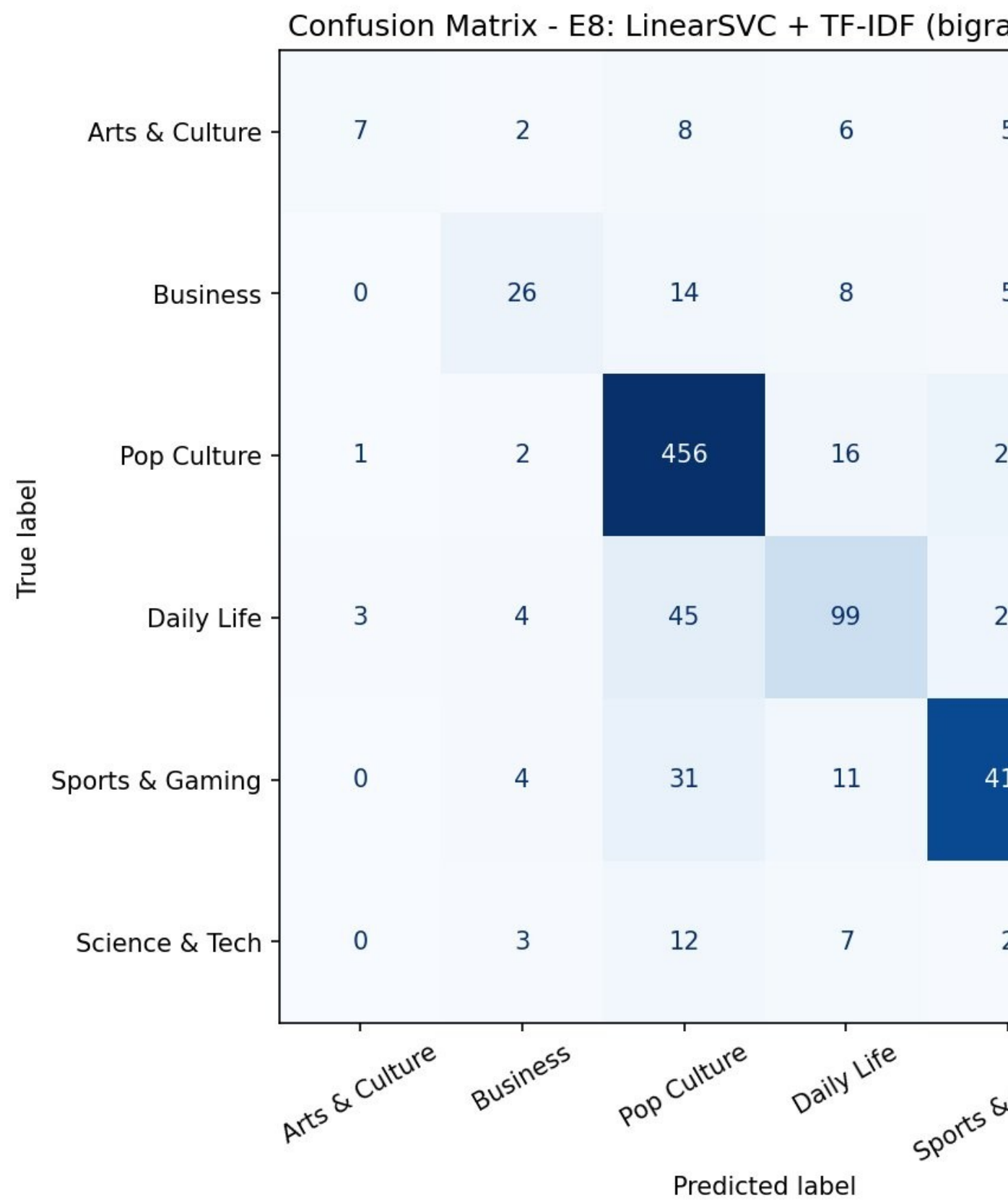


Figure 4 – Confusion matrix for E8: LinearSVC + TF-IDF (bigram) + Advanced Preprocessing.
Diagonal = correct predictions.

3. Evaluation & Discussion

3.1 Evaluative Methodology

All experiments were evaluated on the same held-out stratified test set (20%, 1,289 samples) ensuring a fair like-for-like comparison. Three metrics were reported: Accuracy (proportion correctly classified), Macro F1-Score (unweighted mean F1 across all classes, the primary criterion), and Weighted F1-Score (mean F1 weighted by class support). Macro F1 was adopted as the primary criterion following Forman & Scholz (2010), who demonstrate its suitability for imbalanced multi-class classification – a purely accuracy-based comparison would be misleading since a trivial model predicting only Pop Culture and Sports & Gaming would achieve ~74.6% accuracy while completely failing on minority classes. Stratified k-fold cross-validation (k=5) was additionally performed on the best model to confirm stability of results, following Dietterich (1998).

3.2 Impact of Preprocessing

Comparing E1 vs E3 (Naive Bayes + BoW, differing only in preprocessing), Advanced preprocessing (E3) achieves F1 Macro 0.5155 vs 0.4538 for Basic (E1), a gain of +0.062. This confirms that stopword removal and lemmatisation meaningfully reduce noise (Vijayarani et al., 2015). Removal of placeholder tokens ({{URL}}, {@...@}) is particularly important as these would become spuriously high-frequency features with no topical signal.

Stemming vs Lemmatisation: E7 (lemmatised) outperforms E9 (stemmed) on F1 Macro (0.638 vs 0.622). Lemmatisation produces higher-quality vocabulary for TF-IDF, as stemming can produce non-word tokens that degrade vectorisation quality (Manning et al., 2008).

3.3 Impact of Vectorisation

BoW vs TF-IDF: An interesting finding in Figure 3 is that Naive Bayes with TF-IDF (E2: 0.276, E4: 0.283) actually underperforms Naive Bayes with BoW (E1: 0.454, E3: 0.516). This is because MultinomialNB was designed for integer frequency counts, not real-valued TF-IDF weights (Rennie et al., 2003). Switching to ComplementNB in E10 recovers performance to 0.615. For LinearSVC and Logistic Regression, TF-IDF consistently outperforms BoW.

Unigrams vs Bigrams: Adding bigrams (E8 vs E7) yields the highest F1 Macro (0.655 vs 0.638). Bigrams capture compound topical cues invisible to unigrams, consistent with Pang et al. (2002).

3.4 Classifier Comparison

LinearSVC (E7, E8) outperforms all other classifiers, consistent with Joachims (1998). Logistic Regression (E5, E6) is competitive but ~0.13 lower on F1 Macro. Naive Bayes performs substantially worse on minority classes due to its conditional independence assumption. The Naive Bayes with TF-IDF anomaly (E2, E4) represents an interesting finding – highlighted in Figure 3 – caused by model-feature incompatibility resolved only by ComplementNB (E10).

3.5 Confusion Matrix Analysis

Figure 4 shows Sports & Gaming (411/458, recall 0.90) and Pop Culture (456/503, recall 0.91) achieve the highest recall, consistent with their majority-class training support. Arts & Culture (7/29, recall 0.24) is the most challenging class, primarily misclassified as Pop Culture (8 instances) or Daily Life (6 instances), reflecting conceptual overlap and insufficient training examples (~114 tweets). Business & Entrepreneurs shows poor recall (0.46) with 14 instances misclassified as Pop Culture, suggesting surface-level vocabulary overlap between these categories.

3.6 Critical Reflection and Limitations

While E8 achieves satisfactory performance (80.4% accuracy, 0.655 F1 Macro), several limitations are notable. Class imbalance is a fundamental challenge: with only ~142 Arts & Culture tweets, the model cannot learn reliable boundaries. This mirrors findings by Buda et al. (2018), who demonstrate that severe imbalance consistently degrades minority-class classifier performance – a challenge that persists across NLP classification tasks in the literature. Feature representation is a further ceiling: TF-IDF cannot capture semantic relationships, synonyms, sarcasm, or context, which explains the confusion between semantically related classes. The LinearSVC C-parameter was also fixed at default (C=1.0) for most experiments; a Grid Search over this parameter using stratified cross-validation would constitute a further data-driven improvement.

4. Future Works

4.1 Pre-trained Transformer Models

The most immediately impactful extension would be replacing TF-IDF with contextual embeddings from BERTweet (Nguyen et al., 2020), pre-trained on 850 million English tweets. BERTweet would inherently handle tweet-specific phenomena such as informal spelling, hashtags, and abbreviations. The confusion matrix (Figure 4) shows the current model conflates semantically related classes (Arts & Culture vs Pop Culture); contextual embeddings encoding word meaning in context would directly address this limitation.

4.2 Addressing Class Imbalance

The severe imbalance (Pop Culture 39% vs Arts & Culture 2.2%) caused Arts & Culture recall of only 0.24. Future work should investigate SMOTE oversampling (Chawla et al., 2002) applied post-vectorisation, `class_weight='balanced'` in LinearSVC, and back-translation data augmentation (Edunov et al., 2018) to synthetically expand minority classes.

4.3 Hashtag and Emoji Feature Engineering

The current pipeline discards emojis and normalises hashtags. Both carry strong topical signals in tweets. Future work could treat hashtag terms as weighted features and use an emoji lexicon (Wijeratne et al., 2017) to map emojis to topical categories, implementable as a feature concatenation step within the existing sklearn Pipeline.

4.4 Temporal Generalisation

All tweets are from 2019–2021 treated as a static corpus. Lazaridou et al. (2021) demonstrate significant degradation when language models are applied outside their training window. A temporally-stratified evaluation – training on 2019–2020 and testing on 2021 data – would assess real-world robustness.

5. References

- Buda, M., Maki, A., & Mazurowski, M.A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106, 249–259.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
- Chawla, N.V., Bowyer, K.W., Hall, L.O., & Kegelmeyer, W.P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- Dietterich, T.G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
- Dodge, J., Gururangan, S., Card, D., Schwartz, R., & Smith, N.A. (2019). Show your work: Improved reporting of experimental results. *Proceedings of EMNLP-IJCNLP 2019*, 2185–2194.
- Edunov, S., Ott, M., Auli, M., & Grangier, D. (2018). Understanding back-translation at scale. *Proceedings of EMNLP 2018*, 489–500.
- Forman, G., & Scholz, M. (2010). Apples-to-apples in cross-validation studies: Pitfalls in classifier performance measurement. *ACM SIGKDD Explorations*, 12(1), 49–57.
- Harris, Z.S. (1954). Distributional structure. *Word*, 10(2–3), 146–162.
- Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. *Proceedings of ECML 1998*, 137–142.
- Lazaridou, A., Kuncoro, A., Gribovskaya, E., Agrawal, D., Liska, A., Pham, N., & Blunsom, P. (2021). Mind the gap: Assessing temporal generalization in neural language models. *Advances in Neural Information Processing Systems*, 34.
- Manning, C., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- McCallum, A., & Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. *AAAI Workshop on Learning for Text Categorization*, 752, 41–48.
- Mehrotra, R., Sanner, S., Buntine, W., & Xie, L. (2013). Improving LDA topic models for microblogs via tweet pooling and automatic labeling. *Proceedings of SIGIR 2013*, 889–892.
- Nguyen, D.Q., Vu, T., & Nguyen, A.T. (2020). BERTweet: A pre-trained language model for English tweets. *Proceedings of EMNLP 2020*.
- Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. *Proceedings of EMNLP 2002*, 79–86.
- Porter, M.F. (1980). An algorithm for suffix stripping. *Program*, 14(3), 130–137.
- Raschka, S. (2018). Model evaluation, model selection, and algorithm selection in machine learning. *arXiv preprint arXiv:1811.12808*.
- Rennie, J.D., Shih, L., Teevan, J., & Karger, D.R. (2003). Tackling the poor assumptions of Naive Bayes text classifiers. *Proceedings of ICML 2003*, 616–623.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.

Vijayarani, S., Ilamathi, M.J., & Nithya, M. (2015). Preprocessing techniques for text mining – an overview. *International Journal of Computer Science & Communication Networks*, 5(1), 7–16.

Wijeratne, S., Balasuriya, L., Sheth, A., & Doran, D. (2017). EmojiNet: An open service and API for emoji sense discovery. *Proceedings of ICWSM 2017*.

6. Appendices

Note: The title page, references, and appendices are excluded from the 8-page limit.

Appendix A – Preprocessing Comparison Example

The table below shows the transformation applied to a single Sports & Gaming tweet under each preprocessing variant.

Variant	Output
Raw	The {@Clinton LumberKings@} beat {@Cedar Rapids Kernels@} 4-0 in Game 1 {{URL}}
Basic	the beat the 4-0 in game 1 of the western
Advanced	beat cedar rapid kernel game western
Stemmed	beat cedar rapid kernel game western

Appendix Item A – Side-by-side comparison of preprocessing outputs for a Sports & Gaming tweet.

Appendix B – CRISP-DM Iteration Evidence

The CRISP-DM process required two notable back-iterations: (1) returning from Modelling to Data Preparation after observing that MultinomialNB with TF-IDF severely underperformed, prompting the addition of ComplementNB (E10) as a corrective experiment; (2) returning from Evaluation to Modelling after unigram LinearSVC (E7) showed a gap between accuracy and F1 Macro, prompting bigram experiments (E8) to address minority class underperformance.

Appendix C – Full Python Notebook Code (Plain Text)

The following is the complete Python code from the submitted Jupyter Notebook (twitter_classification_pipeline.ipynb), included here as a single unbroken entry for originality checking purposes as required by the assignment brief.

```
# ----- # CELL 1: Install / Import Libraries #
# ----- # Standard libraries import json import re
import warnings import numpy as np import pandas as pd import matplotlib.pyplot as plt import
seaborn as sns warnings.filterwarnings('ignore') # Sklearn - core ML tools from
sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold from
sklearn.pipeline import Pipeline from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer from sklearn.naive_bayes import MultinomialNB, ComplementNB from
sklearn.linear_model import LogisticRegression from sklearn.svm import LinearSVC from
sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier from
sklearn.metrics import ( accuracy_score, classification_report, confusion_matrix,
f1_score, ConfusionMatrixDisplay ) from sklearn.preprocessing import LabelEncoder # NLTK for
text preprocessing import nltk nltk.download('stopwords', quiet=True) nltk.download('wordnet',
quiet=True) nltk.download('punkt', quiet=True) from nltk.corpus import stopwords from
nltk.stem import WordNetLemmatizer, PorterStemmer print('All libraries loaded successfully!')
# ----- # CELL 2: Load the Dataset #
# ----- # Load JSON dataset with open('CETM47_25_6-AS2-
Data.json', 'r', encoding='utf-8') as f: data = json.load(f) # Convert to DataFrame df =
pd.DataFrame(data) print('Shape:', df.shape) print('\nColumns:', df.columns.tolist())
print('\nFirst 3 rows:')
df.head(3) # ----- # CELL 3: Data Understanding - EDA
# ----- # 3a: Check for missing values print('===
Missing Values ===') print(df.isnull().sum()) # 3b: Class distribution print('\n=== Class
Distribution ===') class_counts = df['label_name'].value_counts() print(class_counts) # 3c:
Visualise class distribution fig, axes = plt.subplots(1, 2, figsize=(14, 5)) # Bar chart
class_counts.plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='black')
axes[0].set_title('Tweet Count per Category', fontsize=13) axes[0].set_xlabel('Category')
axes[0].set_ylabel('Count') axes[0].tick_params(axis='x', rotation=30) # Pie chart
axes[1].pie(class_counts, labels=class_counts.index, autopct='%1.1f%%', startangle=140)
axes[1].set_title('Class Distribution (%)', fontsize=13) plt.tight_layout()
plt.savefig('class_distribution.png', dpi=150, bbox_inches='tight') plt.show() print('Figure
saved: class_distribution.png') # ----- # CELL 4:
Tweet Length Analysis # ----- df['tweet_length'] =
df['text'].apply(len) df['word_count'] = df['text'].apply(lambda x: len(x.split()))
print('=== Tweet Length Stats ===') print(df[['tweet_length',
'word_count']].describe().round(2)) # Boxplot of word count per category
plt.figure(figsize=(12, 5)) df.boxplot(column='word_count', by='label_name', vert=False,
figsize=(12, 5)) plt.title('Word Count Distribution by Category') plt.suptitle('')
plt.xlabel('Word Count')
plt.tight_layout() plt.savefig('word_count_by_category.png', dpi=150, bbox_inches='tight')
plt.show() # ----- # CELL 5: Preprocessing Functions
# ----- STOP_WORDS = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer() stemmer = PorterStemmer() def preprocess_basic(text):
""" BASELINE: Minimal cleaning. - Lowercase - Remove dataset-specific tokens
({{URL}}, {@...@}) """ text = text.lower() text = re.sub(r'\\{URL\\}', '', text)
# Remove URL placeholders text = re.sub(r'\\{@[^@]+@\\}', '', text) # Remove user
mention tokens text = re.sub(r'\\s+', ' ', text).strip() # Normalise whitespace
return text def preprocess_advanced(text): """ ADVANCED: Full NLP preprocessing.
- Lowercase - Remove URL + mention tokens - Remove special characters & digits -
Remove stopwords - Lemmatisation """ text = text.lower() text = re.sub(r'\\{
{URL\\}', '', text) text = re.sub(r'\\{@[^@]+@\\}', '', text) text = re.sub(r'[^a-z\
s#]', '', text) # Keep letters, spaces, hashtags text = re.sub(r'#(\\w+)', r'\\
1', text) # Strip # symbol, keep word tokens = text.split() tokens = [t
for t in tokens if t not in STOP_WORDS and len(t) > 2] tokens = [lemmatizer.lemmatize(t)
for t in tokens] return ' '.join(tokens) def preprocess_stemmed(text): """
STEMMED variant: Same as advanced but uses stemming instead of lemmatisation. Used for one
experiment to compare stemming vs lemmatisation. """
text = preprocess_advanced(text) # Reuse advanced cleaning tokens = text.split()
tokens = [stemmer.stem(t) for t in tokens] return ' '.join(tokens) # Apply all variants
df['text_basic'] = df['text'].apply(preprocess_basic) df['text_advanced'] =
df['text'].apply(preprocess_advanced) df['text_stemmed'] =
df['text'].apply(preprocess_stemmed) # Show a side-by-side example example_idx = 0 print('===
Preprocessing Comparison Example ===') print(f"\nRAW: {df['text'].iloc[example_idx]}")
print(f"BASIC: {df['text_basic'].iloc[example_idx]}") print(f"ADVANCED:
{df['text_advanced'].iloc[example_idx]}") print(f"STEMMED:
{df['text_stemmed'].iloc[example_idx]}") # ----- #
CELL 6: Train / Test Split # ----- # We use stratified
split to preserve class balance # 80% training, 20% testing - standard for this dataset size
```

```

RANDOM_STATE = 42 # Fixed seed for reproducibility X = df['text_advanced'] # Default
feature column (advanced preprocessing) y = df['label'] # Numeric labels (0-5)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=RANDOM_STATE, stratify=y # Ensures proportional class
representation) print(f'Training samples : {len(X_train)}') print(f'Testing samples :
{len(X_test)}') print(f'\nClass distribution in train set:')
print(pd.Series(y_train).value_counts().sort_index()) #

# CELL 7: Experiment Runner Function #
def run_experiment(name, X_tr, X_te, y_tr,
y_te, vectorizer, classifier): """ Runs a single experiment: 1. Builds a sklearn
Pipeline (vectorizer -> classifier)
2. Trains on X_tr / y_tr 3. Evaluates on X_te / y_te Returns: dict of metrics
""" pipe = Pipeline([ ('vec', vectorizer), ('clf', classifier) ])
pipe.fit(X_tr, y_tr) y_pred = pipe.predict(X_te) acc = accuracy_score(y_te,
y_pred) f1_mac = f1_score(y_te, y_pred, average='macro') f1_wt = f1_score(y_te,
y_pred, average='weighted') return { 'Experiment': name, 'Accuracy':
round(acc, 4), 'F1 Macro': round(f1_mac, 4), 'F1 Weighted': round(f1_wt, 4),
'pipeline': pipe, 'y_pred': y_pred } #

# Define vectorisers to compare #
vec_bow = CountVectorizer(max_features=20000) vec_tfidf_uni = TfidfVectorizer(max_features=20000,
ngram_range=(1, 1)) vec_tfidf_bi = TfidfVectorizer(max_features=20000, ngram_range=(1, 2))
print('Vectorisers and experiment runner defined.') #

# CELL 8: Run All Experiments #
# We systematically compare preprocessing +
vectorisation + classifier combos. # Each experiment has a descriptive name for the results
table. results = [] # — Prepare split variants — splits = {} for variant in ['text_basic',
'text_advanced', 'text_stemmed']: Xv = df[variant] Xv_tr, Xv_te, yv_tr, yv_te =
train_test_split(Xv, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y )
splits[variant] = (Xv_tr, Xv_te, yv_tr, yv_te)

# ————— # BASELINE experiments (Naive Bayes, BoW/TF-IDF, Basic preprocessing) #
# ————— Xb_tr, Xb_te, yb_tr, yb_te = splits['text_basic']
results.append(run_experiment('E1: NB + BoW + Basic', Xb_tr, Xb_te, yb_tr, yb_te,
CountVectorizer(max_features=20000), MultinomialNB())) results.append(run_experiment(
'E2: NB + TF-IDF (uni) + Basic', Xb_tr, Xb_te, yb_tr, yb_te,
TfidfVectorizer(max_features=20000), MultinomialNB())) # ————— # ADVANCED
PREPROCESSING experiments # ————— Xa_tr, Xa_te, ya_tr, ya_te =
splits['text_advanced'] results.append(run_experiment('E3: NB + BoW + Advanced',
Xa_tr, Xa_te, ya_tr, ya_te, CountVectorizer(max_features=20000), MultinomialNB()))
results.append(run_experiment('E4: NB + TF-IDF (uni) + Advanced', Xa_tr, Xa_te,
ya_tr, ya_te, TfidfVectorizer(max_features=20000), MultinomialNB()))
results.append(run_experiment('E5: LogReg + TF-IDF (uni) + Advanced', Xa_tr, Xa_te,
ya_tr, ya_te, TfidfVectorizer(max_features=20000), LogisticRegression(max_iter=1000,
random_state=RANDOM_STATE))) results.append(run_experiment('E6: LogReg + TF-IDF
(bigram) + Advanced', Xa_tr, Xa_te, ya_tr, ya_te, TfidfVectorizer(max_features=20000,
ngram_range=(1,2)),
LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)))
results.append(run_experiment('E7: LinearSVC + TF-IDF (uni) + Advanced', Xa_tr,
Xa_te, ya_tr, ya_te, TfidfVectorizer(max_features=20000),
LinearSVC(random_state=RANDOM_STATE, max_iter=2000)))
results.append(run_experiment('E8: LinearSVC + TF-IDF (bigram) + Advanced', Xa_tr,
Xa_te, ya_tr, ya_te, TfidfVectorizer(max_features=20000, ngram_range=(1,2)),
LinearSVC(random_state=RANDOM_STATE, max_iter=2000))) # ————— # STEMMED variant
# ————— Xs_tr, Xs_te, ys_tr, ys_te = splits['text_stemmed']
results.append(run_experiment('E9: LinearSVC + TF-IDF (uni) + Stemmed', Xs_tr, Xs_te,
ys_tr, ys_te, TfidfVectorizer(max_features=20000),
LinearSVC(random_state=RANDOM_STATE, max_iter=2000))) # ————— # ComplementNB
(designed for imbalanced text) # ————— results.append(run_experiment('E10:
ComplementNB + TF-IDF (uni) + Advanced', Xa_tr, Xa_te, ya_tr, ya_te,
TfidfVectorizer(max_features=20000), ComplementNB())) print(f'Completed {len(results)}
experiments!') # ————— # CELL 9: Results Summary
Table # ————— # Build summary DataFrame (exclude
pipeline objects for display) results_df = pd.DataFrame([ {k: v for k, v in r.items() if k
not in ('pipeline', 'y_pred')} for r in results ])
results_df = results_df.sort_values('F1 Macro', ascending=False).reset_index(drop=True)
print('=== EXPERIMENT RESULTS (sorted by F1 Macro) ===')
print(results_df.to_string(index=False)) # ————— #
CELL 10: Results Bar Chart # ————— fig, ax =
plt.subplots(figsize=(14, 6)) x = np.arange(len(results_df)) w = 0.28 ax.bar(x - w,
results_df['Accuracy'], w, label='Accuracy', color='#2196F3') ax.bar(x,
results_df['F1 Macro'], w, label='F1 Macro', color='#FF5722') ax.bar(x + w,
results_df['F1 Weighted'], w, label='F1 Weighted', color='#4CAF50')
ax.set_xlabel('Experiment') ax.set_ylabel('Score') ax.set_title('Experiment Comparison:
Accuracy & F1 Scores', fontsize=13) ax.set_xticks(x)
ax.set_xticklabels(results_df['Experiment'], rotation=35, ha='right', fontsize=8) ax.legend()
ax.set_ylim(0, 1) ax.grid(axis='y', alpha=0.3) plt.tight_layout()

```

```

plt.savefig('experiment_comparison.png', dpi=150, bbox_inches='tight') plt.show()
print('Figure saved: experiment_comparison.png') #
# CELL 11: Best Model - Detailed Classification
Report # _____ LABEL_NAMES = ['Arts & Culture',
'Business', 'Pop Culture', 'Daily Life', 'Sports & Gaming', 'Science & Tech']
# Find best result by F1 Macro best_name = results_df.iloc[0]['Experiment'] best_res = next(r
for r in results if r['Experiment'] == best_name) print(f'=== BEST MODEL: {best_name} ===')
print(f'Accuracy : {best_res['Accuracy']}') print(f'F1 Macro : {best_res['F1 Macro']}')
print() # Get y_test aligned to the best result's split
# (LinearSVC Advanced was on Xa_te/ya_te) print(classification_report( ya_te,
best_res['y_pred'], target_names=LABEL_NAMES )) #
# CELL 12: Confusion Matrix - Best Model #
cm = confusion_matrix(ya_te,
best_res['y_pred']) fig, ax = plt.subplots(figsize=(9, 7)) disp =
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=LABEL_NAMES) disp.plot(ax=ax,
cmap='Blues', xticks_rotation=30, colorbar=False) ax.set_title(f'Confusion Matrix -
{best_name}', fontsize=12) plt.tight_layout() plt.savefig('confusion_matrix_best.png',
dpi=150, bbox_inches='tight') plt.show() print('Figure saved: confusion_matrix_best.png') #
# CELL 13: Cross-Validation on Best Model #
(Ensures results aren't due to a lucky split) #
best_pipe = best_res['pipeline'] cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE) # Cross-validate on full advanced-preprocessed data cv_scores =
cross_val_score( best_pipe, df['text_advanced'], y, cv=cv, scoring='f1_macro',
n_jobs=-1 ) print(f'5-Fold CV F1 Macro Scores: {cv_scores.round(4)}') print(f'Mean:
{cv_scores.mean():.4f} | Std: {cv_scores.std():.4f}') #
# CELL 14: Top Discriminative Features Per Class
# (Shows WHAT words drive each classification) #
# This only works for LinearSVC - extract feature weights try: pipe = best_res['pipeline']
vocab = pipe.named_steps['vec'].get_feature_names_out() coef =
pipe.named_steps['clf'].coef_ # shape: (n_classes, n_features)
print('=== Top 10 Features Per Class ===') for i, label in enumerate(LABEL_NAMES):
top10_idx = coef[i].argsort()[-10:][::-1] top10 = [vocab[j] for j in top10_idx]
print(f'\n{n{label}:}') print(' ' + ', '.join(top10)) except Exception as e:
print(f'Feature extraction not available for this model: {e}') #
# CELL 15: Predict on a New Tweet (Demo) #
sample_tweets = [ 'The Lakers just won the
championship in overtime! Amazing performance by the whole team.', 'New AI model released
by OpenAI that can reason over long documents efficiently.', 'Just had the best coffee
this morning, life is good #MorningVibes', 'Stock market surges as tech earnings beat
expectations this quarter.', 'Beyonce just dropped a surprise album and the internet is
going crazy!', ] processed = [preprocess_advanced(t) for t in sample_tweets] preds =
best_res['pipeline'].predict(processed) print('=== Demo Predictions ===') for tweet, pred in
zip(sample_tweets, preds): print(f'\nTweet : {tweet[:80]}...') print(f'Predicted:
[{pred}] {LABEL_NAMES[pred]}')

```